MASTER'S THESIS PROPOSAL

Code Embeddings for JavaScript Bundle Composition Analysis

Rabie Alawad

Supervisor: Dr. Ben Swierzy · University of Bonn

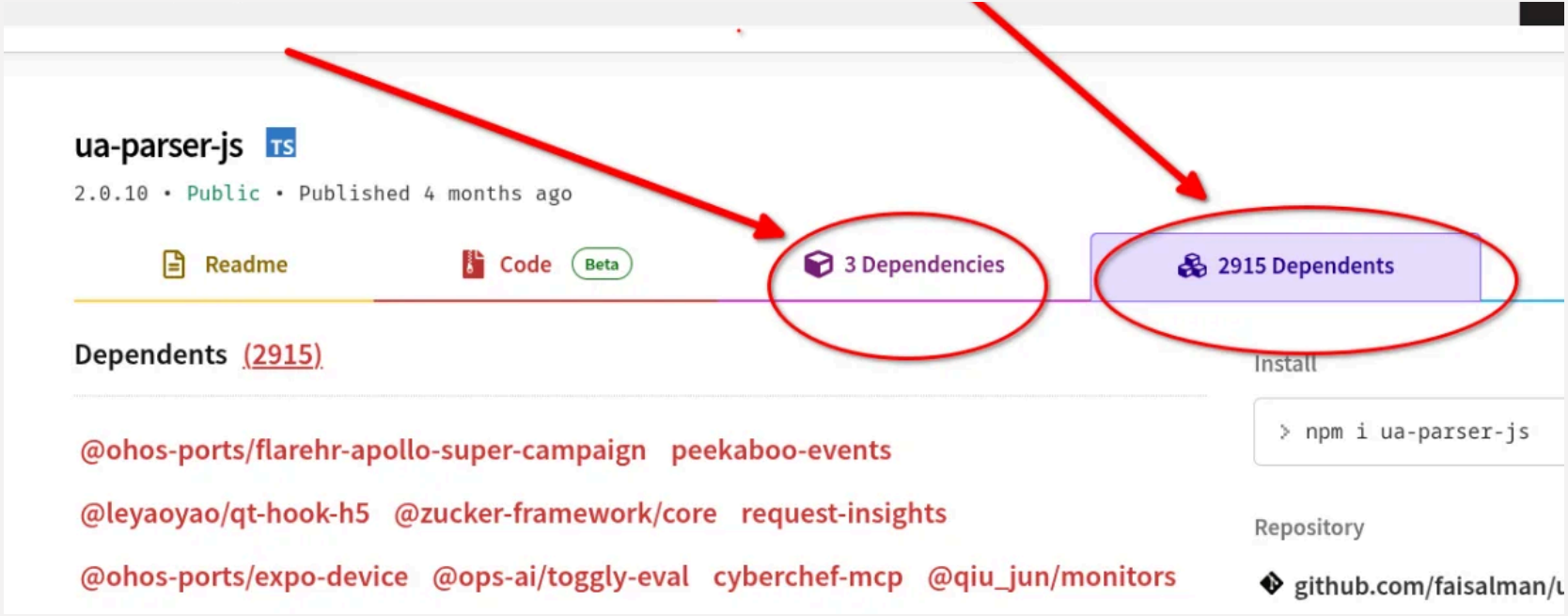
MOTIVATION

Bundlers Erase the Dependency Manifest

- Modern websites ship hundreds of third-party npm packages.
- A vulnerability in any package is inherited by every site using it.
- Bundlers minify and obscure the dependency manifest.

MOTIVATION

Hundreds of Transitive Dependencies



MOTIVATION

Minification Erases Identity

Source (react-dom@18.2.0)

```
import { createRoot } from
"react-dom/client";

function App() {
  return <h1>Hello</h1>;
}

createRoot(root).render(<App
/>);
```

Minified bundle output

```
function _(){return
e.createElement("h1",null,"Hello")}t.render(e.createElement(_,null),n)
```

Package names, versions and boundaries are gone — no manifest survives minification.

MOTIVATION

5–20%

of production websites update their dependencies within 16 weeks of a patch.

MOTIVATION

Aletheia Recovers Versions via Rolling AST Hashes

87%

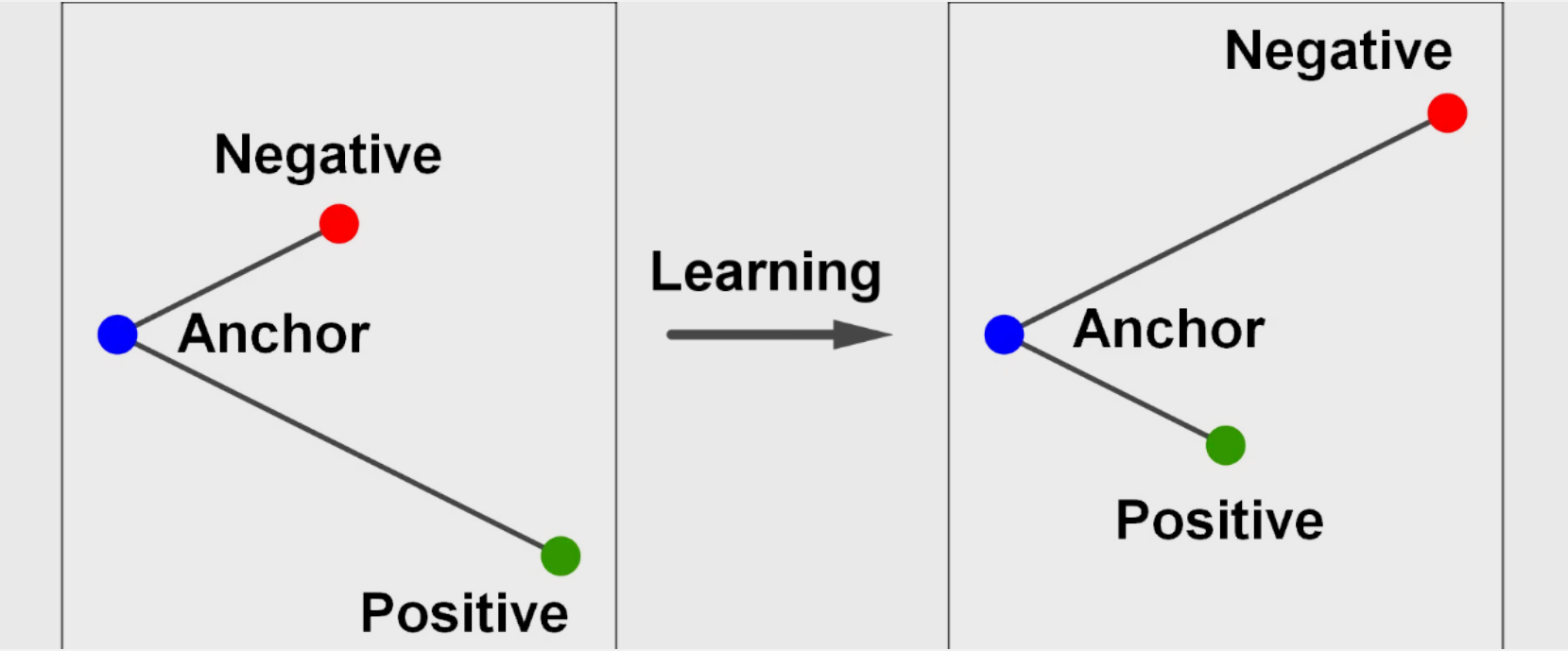
patch-level detection in the lab

82%

patch-level detection on real-world bundles

Remaining errors concentrate in small changesets that exact hash overlap cannot resolve.

Embeddings of Similar Instances Can be Pushed Apart



Triplet loss pulls matching pairs closer, pushes non-matches apart

PART TWO

Research Questions

RESEARCH QUESTIONS

Four Questions Guide the Work

RQ1

Which code representations stay stable under bundler transforms: minification, tree shaking, inlining?

RQ2

When do embeddings beat non-learned approaches like hashing?

RQ3

Can embeddings identify packages in bundles without source maps, currently an open problem?

RQ4

What trade-offs exist between accuracy and computational cost?

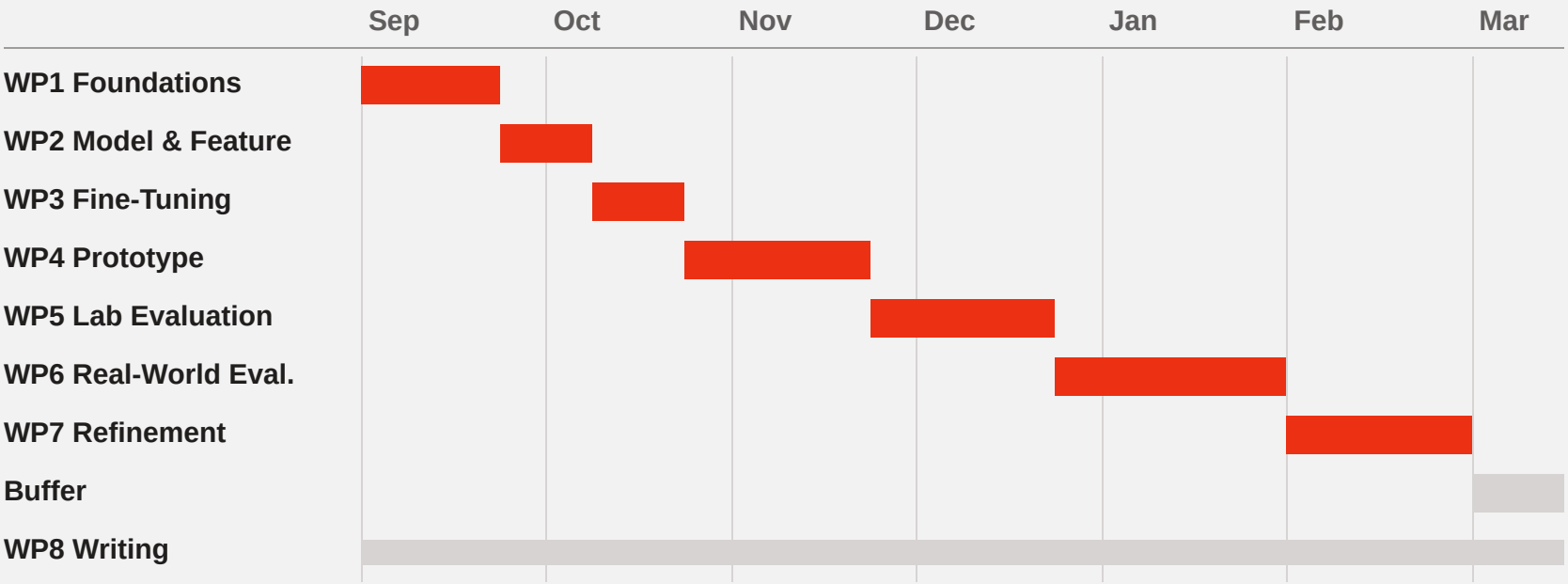
PART THREE

Research Plan

6 months · 26 weeks · September – March

RESEARCH PLAN

Timeline — 26 Weeks



WORK PACKAGES

WP1–4

WP1 — Foundations

Survey embedding-based matching in binary SCA tools

WP2 — Model & Features

Test embedding models and code representations

WP3 — Fine-Tuning

Test fine-tuning approaches: contrastive, triplet-loss, margin-based

WP4 — Prototype

Assemble pipeline and profile cost early

WORK PACKAGES

WP5–8

WP5 — Lab Evaluation

Evaluate on bundles with controlled ground truth

WP6 — Real-World Eval

Evaluate on production bundles and analyze failures

WP7 — Refinement & Buffer

Tune from error analysis and optimize index

WP8 — Writing

Write continuously: related work, methods, results, discussion

Questions & Discussion

Rabie Alawad · University of Bonn